

AI 协作反思日志 — Phase 2 体系结构设计

团队：第44组

日期：2026年5月3日

1. 本阶段 AI 使用清单

任务	AI工具	Prompt摘要	AI输出质量(1-5)	人工修改幅度(%)
架构风格对比与选择	豆包/千问	分别采用4种策略咨询架构建议：①直接询问推荐架构；②提供"4人学生团队、10周工期、无Web经验"约束后询问；③要求对比MVC分层与模块化垂直架构优劣；④让AI扮演课程项目架构师分析	3	35%
模块划分与接口定义	Kimi	基于选定的模块化垂直架构，生成4大模块（用户权限、任务管理、标注核心、冲突裁定）的详细职责边界、核心接口、数据表结构及模块依赖关系	4	30%
技术选型方案	Kimi	生成完整技术选型表，要求包含前端框架、后端框架、数据库、缓存、部署方式等层次，并给出备选方案及排除原因	4	25%
ADR编写	千问	按ADR模板生成4条架构决策记录（模块化垂直架构、Vue3+Spring Boot前后端分离、MySQL单库存储、JWT无状态认证），要求包含背景、决策、理由、后果	3	40%
架构设计文档整合	DeepSeek	整合模块划分、技术选型、ADR为完整架构设计文档，要求包含架构概览、模块划分说明、技术选型说明、ADR集合	4	25%

2. AI 助力分析 (Q1)

本阶段 AI 最大的贡献是：快速生成ADR的标准格式框架、模块划分的初始结构以及技术选型的对比表格，无需从零搭建文档骨架，尤其ADR模板中的"背景-决策-理由-后果"结构直接可用；同时AI在生成技术选型的"备选方案及排除原因"时提供了较全面的企业级技术视野，为人工判断提供了参照系。

不同 Prompt 策略下 AI 回答质量的差异：

- 策略A（直接问）：AI推荐经典MVC三层架构+微服务风格，技术栈偏向企业级（建议Redis缓存、MongoDB、OAuth2、Docker+K8s），完全忽略学生团队约束，输出质量评分2分，人工需大幅修正。
- 策略B（提供详细约束后问）：AI输出明显贴近课程场景，开始提及"低耦合协作"、"并行开发"，技术栈收敛至Vue+Spring Boot，但仍默认推荐Redis和MyBatis-Plus，输出质量评分4分。
- 策略C（要求对比多种方案）：AI能列出MVC与垂直架构的对比维度（开发复杂度、可维护性、协作效率），但对"团队学习成本"和"10周可行性"的量化分析不足，输出质量评分3分。
- 策略D（让AI扮演架构师角色）：AI输出最符合课程答辩语境，主动提及"学生易掌握"、"利于演示展示"、"降低失败风险"等课程特定考量，输出质量评分4分。

结论：提供详细项目约束+角色扮演的组合Prompt策略效果最佳，直接询问的Prompt几乎不可用。

3. AI 误导分析 (Q2)

AI 在架构建议中的典型问题：

- 问题1：架构风格推荐偏离课程场景 —— AI初稿推荐经典MVC三层架构，侧重代码复用和逻辑分层，未考虑4人学生团队在无Web开发经验下的协作冲突风险。

如何发现：对照P1阶段已出现的"角色权限分配错误、术语误用"等协作阻塞问题，以及P2任务要求中"10周工期、无Web开发经验"的硬约束，发现MVC水平分层会导致多人频繁修改同一

Controller/Service层，Git冲突成本极高。

如何修正：改为**模块化垂直架构**，按业务功能垂直切分（用户权限→任务管理→标注核心→冲突裁定），每人负责完整模块的前端+后端+数据库，模块间通过API轻量交互，以牺牲部分代码复用性换取并行开发效率最大化。

- **问题2：**技术选型过度设计，忽视学生运维能力边界 —— AI初稿在技术选型中默认推荐Redis缓存、MongoDB、OAuth2、Docker+K8s等方案，将"可扩展性"置于"可交付性"之上。

如何发现：对照团队技术背景、部署环境、数据规模，发现引入Redis/MongoDB属于过度设计，且团队缺乏分布式运维经验，10周内无法完成环境搭建与调优。

如何修正：明确采用MySQL 8.0单库存储，不引入Redis/NoSQL；采用服务器本地文件系统+Nginx处理静态资源；保持架构极简，确保可交付。

- **问题3：**ADR中遗漏关键质量属性的权衡分析 —— AI初稿将JWT描述为"完美方案"，将MySQL描述为直接建议，只陈述技术优势，未提及安全性风险、扩展性局限等负面影响。

如何发现：对照软件工程课程中"架构决策必须明确trade-off"的要求，以及P1阶段AI"将界面拖拽缩放标注为Must"的优先级误判教训，发现AI在ADR的"后果"章节中几乎只写正面影响，忽略负面影响与风险。

如何修正：人工强制补充每条ADR的负面影响与风险：

- JWT：补充"Token有效期内无法强制注销"的负面影响，增加"需防范XSS攻击，加密存储Token"的风险提示；
 - MySQL单库：补充"未来扩展可能受限"的负面影响，增加"需定期备份数据"的风险提示；
 - 模块化垂直架构：补充"公共工具可能重复实现"的负面影响，增加"前期需统一API格式和异常处理"的风险提示。
-

4. Prompt 改进计划 (Q3)

上阶段 (P1) 提出的改进措施, 验证结果:

P1改进措施	验证结果
Prompt中加入"系统核心规则前置说明"	有效。在P2的模块划分Prompt中前置"全局角色+任务级身份双轨制"规则后, AI生成的接口权限设计准确率显著提升, 角色误分配问题减少。
明确输出格式要求 (优先级划分标准、术语规范)	有效。在技术选型Prompt中明确要求"包含备选方案及排除原因"后, AI输出结构更完整, 减少了格式类返工。
采用"分步Prompt"策略 (先框架后内容)	有效。先生成ADR框架确认无误后再填充内容, 避免了P1阶段"整体偏差导致全文重写"的问题。
在Prompt中加入"对照SRS核心约束"的要求	部分有效。AI能对照显式约束 (如四阶段不可逆), 但对隐式约束 (如"数据一致性>操作便捷性"的优先级) 仍无法自主推导。

本阶段新的改进措施:

- 前置"质量属性优先级"声明: 在架构类Prompt中明确声明"本项目的质量属性优先级为: 开发效率 > 可扩展性, 确定性 > 前沿性, 可交付性 > 完美性", 强制AI在推荐方案时按此优先级排序, 避免过度设计。
- 强制Trade-off输出: 在ADR类Prompt中增加硬性要求: "必须在'后果'章节中列出至少1条负面影响和1条风险, 否则输出不合格", 弥补AI倾向于只谈优势的偏差。
- "课程项目架构师"角色固化: 经P2验证, 角色扮演Prompt效果最佳。下阶段统一采用"你是一位有20年经验的课程项目指导教师, 面向4人学生团队, 10周工期, 无生产环境运维需求"的角色设定, 替代通用的"软件架构师"角色。
- 增加"学生团队失败模式"追问: 在技术选型Prompt末尾追加追问: "请列出该方案在学生团队环境下最可能失败的3个环节及规避措施", 利用AI的枚举能力提前暴露风险点。

5. 本阶段的核心工程判断

决策内容：确定采用"模块化垂直架构 + Vue3/Spring Boot前后端分离 + MySQL单库存储 + JWT无状态认证"的整体架构方案。核心权衡在于：明确牺牲代码复用性和未来扩展性，以换取4人并行开发效率最大化；明确不引入Redis、MongoDB、微服务、K8s等任何中间件，保持架构极简以确保10周内可交付；明确"四阶段标注流程不可逆"为系统级约束，不设置撤销/重做功能，优先保证标注数据的严肃性与一致性。

为什么 AI 无法做这个决策：

该决策需要综合权衡多维度非技术性因素，AI无法完成此类隐性推理：

1. **无法结合团队历史痛点进行推理：**AI不知道我们P1阶段已出现过"角色权限分配错误导致返工"的协作阻塞，因此无法理解"模块化垂直架构"的本质动机是**降低认知偏差成本**，而非单纯的技术优劣。
2. **无法评估"学生团队运维失败风险"：**AI倾向于推荐企业级最佳实践（Redis、Docker、K8s），但无法判断"10周工期内，一个无运维经验的学生团队引入Redis后，因配置错误导致项目无法演示"的概率远高于"单库性能不足"的概率。这种**风险概率的隐性判断**需要结合具体团队能力模型。
3. **无法平衡"数据严肃性"与"操作便捷性"：**AI默认会推荐支持撤销/重做、版本控制的友好交互方案，但无法理解法学研究场景下"标注数据一旦提交即不可随意修改"的业务刚性。AI仅能基于输入的规则输出内容，无法结合"法学院真实用户"的学术规范诉求做出"牺牲用户体验以换取数据权威性"的价值判断。
4. **无法判断"可交付性"的绝对优先级：**在课程项目语境下，"能跑起来答辩"的优先级高于"架构优雅"，AI缺乏对"课程验收"这一终极约束的感知，因此会持续推荐更"正确"但更复杂的方案。